



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

LAB EXPERMIENTS

OPERATING SYSTEM - CSA0460

NAME: RASHMITHA SENTHIL KUMAR

REG NO: 192525034

BRANCH: B. TECH AIML

Experiment No: 1

Aim:

To create a new process using fork() system call and display the Process ID (PID) of the current process and its Parent Process ID (PPID) using a C program.

Algorithm:

Step 1: Start

Step 2: Include required header files

Step 3: Create a new process using fork()

Step 4: Check whether process creation is successful

Step 5: If child process, display Child PID and Parent PID

Step 6: If parent process, display Parent PID and Child PID

Step 7: Stop

Source code:

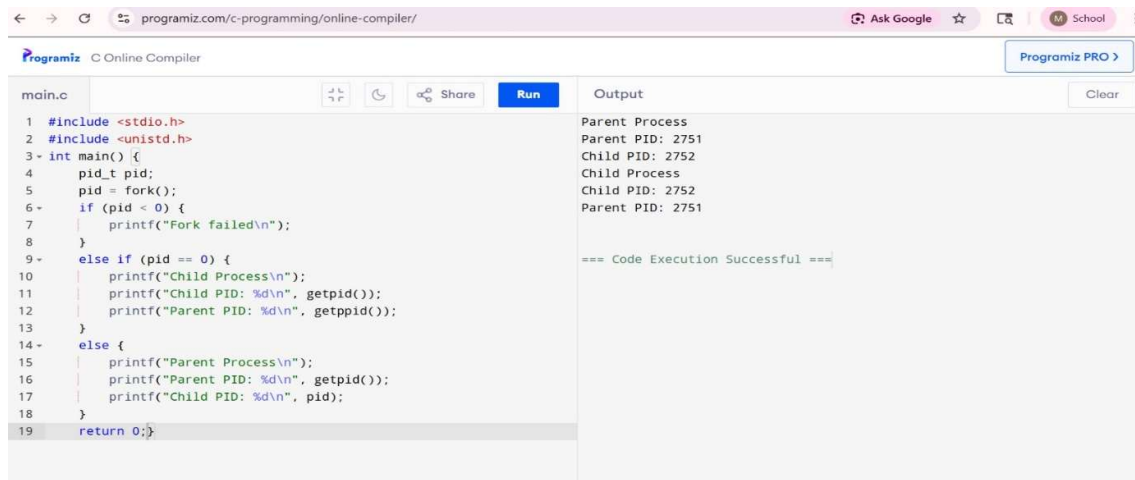
```
#include <stdio.h>
#include <unistd.h>
int main() {
```

```

pid_t pid;
pid = fork()
if (pid < 0) {
    printf("Fork failed\n");
}
else if (pid == 0) {
    printf("Child Process\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());
}
else {
    printf("Parent Process\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);
}
return 0;
}

```

Output:



The screenshot shows a web browser window with the URL `programiz.com/c-programming/online-compiler/`. The page title is "Programiz C Online Compiler". The code editor on the left shows the following C code:

```

1 #include <stdio.h>
2 #include <unistd.h>
3 int main() {
4     pid_t pid;
5     pid = fork();
6     if (pid < 0) {
7         printf("Fork failed\n");
8     }
9     else if (pid == 0) {
10        printf("Child Process\n");
11        printf("Child PID: %d\n", getpid());
12        printf("Parent PID: %d\n", getppid());
13    }
14    else {
15        printf("Parent Process\n");
16        printf("Parent PID: %d\n", getpid());
17        printf("Child PID: %d\n", pid);
18    }
19    return 0;
}

```

The output window on the right shows the following text:

```

Parent Process
Parent PID: 2751
Child PID: 2752
Child Process
Child PID: 2752
Parent PID: 2751

```

Below the output, it says "=== Code Execution Successful ===".

Result: The `fork()` system call was successfully executed. Parent and child processes were created, and their respective Process IDs (PID) and Parent Process IDs (PPID) were displayed correctly.

Experiment No: 2

Aim:

To implement First Come First Serve (FCFS) CPU Scheduling Algorithm and calculate Waiting Time (WT) and Turnaround Time (TAT) for each process.

Algorithm:

Step 1: Start

Step 2: Enter number of processes

Step 3: Enter burst time for each process

Step 4: Set waiting time of first process as 0

Step 5: Calculate waiting time for remaining processes

Step 6: Calculate turnaround time for each process

Step 7: Display Process, BT, WT and TAT

Step 8: Stop

Source code:

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[10], wt[10], tat[10];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
    }
    wt[0] = 0;
```

```

    for(i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
    printf("\nProcess BT WT TAT\n");

    for(i = 0; i < n; i++) {
        printf("P%d %d %d %d\n", i+1, bt[i], wt[i], tat[i]);
    }

    return 0;
}

```

Output:

The screenshot shows the Programiz Online Compiler interface. On the left, the C code is displayed with line numbers. On the right, the output of the program is shown, including the user input and the calculated values for each process.

```

main.c
1 #include <stdio.h>
2 int main() {
3     int n, i;
4     int bt[10], wt[10], tat[10];
5     printf("Enter number of processes: ");
6     scanf("%d", &n);
7     printf("Enter burst time:\n");
8     for(i = 0; i < n; i++) {
9         scanf("%d", &bt[i]);
10    }
11    wt[0] = 0;
12    for(i = 1; i < n; i++) {
13        wt[i] = wt[i-1] + bt[i-1];
14    }
15    for(i = 0; i < n; i++) {
16        tat[i] = wt[i] + bt[i];
17    }
18    printf("\nProcess\tBT\tWT\tTAT\n");
19    for(i = 0; i < n; i++) {
20        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
21    }
22    return 0;
23 }

```

Output:

```

Enter number of processes: 3
Enter burst time:
1 3 5

Process BT WT TAT
P1 1 0 1
P2 3 1 4
P3 5 4 9

=== Code Execution Successful ===

```

Result:

The FCFS CPU Scheduling Algorithm was successfully implemented, and the Waiting Time (WT) and Turnaround Time (TAT) for all processes were calculated correctly.

Experiment no: 3

Aim:

To illustrate the concept of multithreading by creating and executing multiple threads simultaneously using a C program.

Algorithm:

Step 1: Start

Step 2: Include header files

Step 3: Create thread function 1

Step 4: Create thread function 2

Step 5: Create two threads using pthread_create()

Step 6: Execute threads concurrently

Step 7: Wait for completion using pthread_join()

Step 8: Display output

Step 9: Stop

Source code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
// Function executed by Thread 1
void* task1(void* arg)
{
    for(int i = 1; i <= 5; i++)
    {
        printf("Thread 1: %d\n", i);
        sleep(1);
    }
}
```

```

    return NULL;
}
// Function executed by Thread 2
void* task2(void* arg)
{
    for(int i = 1; i <= 5; i++)
    {
        printf("Thread 2: %d\n", i);
        sleep(1);
    }
    return NULL;
}
int main()
{
    pthread_t t1, t2;
    // Create threads
    pthread_create(&t1, NULL, task1, NULL);
    pthread_create(&t2, NULL, task2, NULL);
    // Wait for threads to finish
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Both threads finished execution\n");
    return 0;
}

```

Output:



```

Output
Thread 1: 1
Thread 2: 1
Thread 1: 2
Thread 2: 2
Thread 2: 3
Thread 1: 3
Thread 2: 4
Thread 1: 4
Thread 2: 5
Thread 1: 5
Both threads finished execution

=== Code Execution Successful ===

```

Result:

Multithreading was successfully implemented using pthreads, and multiple threads executed concurrently with proper synchronization.

Experiment no:4

Aim:

To illustrate the concept of inter-process communication using shared memory with a C program.

Algorithm:

Step 1: Start
Step 2: Create shared memory
Step 3: Check memory creation
Step 4: Create child process using fork()
Step 5: Parent writes data into shared memory
Step 6: Child reads data from shared memory
Step 7: Display output
Step 8: Release shared memory
Step 9: Stop

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>

int main() {
    char *shared_memory;

    // Create shared memory
    shared_memory = mmap(NULL, 1024, PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS, -1, 0);
```



```
if (shared_memory == MAP_FAILED) {
    perror("Shared memory creation failed");
    exit(1);
}

// Create child process
pid_t pid = fork();

if (pid < 0) {
    printf("Process creation failed\n");
    exit(1);
}
else if (pid == 0) {
    // Child process reads from shared memory
    sleep(1);
    printf("Child Process Read: %s\n", shared_memory);
}
else {
    // Parent process writes to shared memory
    strcpy(shared_memory, "Hello Child, this is Parent using Shared Memory!");

    printf("Parent Process Wrote: %s\n", shared_memory);

    // Wait for child process
    wait(NULL);

    // Remove shared memory
    munmap(shared_memory, 1024);
}
```

```
    return 0;  
}
```

Output:

Output

Clear

Parent Process Wrote: Hello Child, this is Parent using Shared Memory!
Child Process Read: Hello Child, this is Parent using Shared Memory!

=== Code Execution Successful ===

Result:

Inter-process communication using shared memory was successfully implemented, and data was correctly shared between parent and child processes.

Experiment no: 5

Aim:

Illustrate the concept of inter-process communication using message queue with a C program.

Algorithm:

- Step 1: Start
- Step 2: Define message structure
- Step 3: Generate unique key using ftok()
- Step 4: Create message queue using msgget()
- Step 5: Enter message from user
- Step 6: Send message using msgsnd()
- Step 7: Receive message using msgrcv()
- Step 8: Display received message
- Step 9: Delete message queue
- Step 10: Stop

Source code:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

// Message structure
struct msg_buffer {
    long msg_type;
    char msg_text[100];
```

```
};
```

```
int main() {
```

```
    key_t key;
```

```
    int msgid;
```

```
    struct msg_buffer message;
```

```
    // Generate unique key
```

```
    key = ftok("progfile", 65);
```

```
    // Create message queue
```

```
    msgid = msgget(key, 0666 | IPC_CREAT);
```

```
    printf("Message Queue Created!\n");
```

```
    // ----- SENDER -----
```

```
    message.msg_type = 1;
```

```
    printf("Enter message: ");
```

```
    fgets(message.msg_text, 100, stdin);
```

```
    // Send message
```

```
    msgsnd(msgid, &message, sizeof(message.msg_text), 0);
```

```
    printf("Message sent!\n");
```

```
    // ----- RECEIVER -----
```

```
    msgrcv(msgid, &message, sizeof(message.msg_text), 1, 0);
```

```
    printf("Message received: %s", message.msg_text);
```

```
// Delete message queue  
msgctl(msgid, IPC_RMID, NULL);  
  
return 0;  
}
```

Output:

Output

```
Message Queue Created!  
Enter message: Hello Operating System  
Message sent!  
Message received: Hello Operating System  
  
=== Code Execution Successful ===
```

Result:

Inter-process communication using message queue was successfully implemented, and messages were successfully sent and received between processes.

Experiment No:6

Aim:

To illustrate the concept of deadlock avoidance by simulating Banker's Algorithm using C program.

Algorithm:

- Step 1: Start
- Step 2: Read number of processes and resources
- Step 3: Read Allocation matrix
- Step 4: Read Maximum matrix
- Step 5: Read Available resources
- Step 6: Calculate Need matrix = Max - Allocation
- Step 7: Find process whose Need $\leq$ Available
- Step 8: Add its Allocation to Available and mark process completed
- Step 9: Repeat until all processes complete
- Step 10: If all complete, print safe sequence; otherwise print Unsafe
- Step 11: Stop

Source Code:

```
#include <stdio.h>

int main(){
    int n,m,i,j,k,c=0,f[10]={0},a[10][10],mx[10][10],av[10],nd[10][10],s[10];

    scanf("%d%d",&n,&m);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&a[i][j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&mx[i][j]);

    for(i=0;i<m;i++)
        scanf("%d",&av[i]);
```

```

for(i=0;i<n;i++)
    for(j=0;j<m;j++)
        nd[i][j]=mx[i][j]-a[i][j];

while(c<n){
    int found=0;

    for(i=0;i<n;i++)
        if(!f[i]){
            for(j=0;j<m && nd[i][j]<=av[j];j++);

            if(j==m){
                for(k=0;k<m;k++)
                    av[k]+=a[i][k];

                s[c++]=i;
                f[i]=found=1;
            }
        }

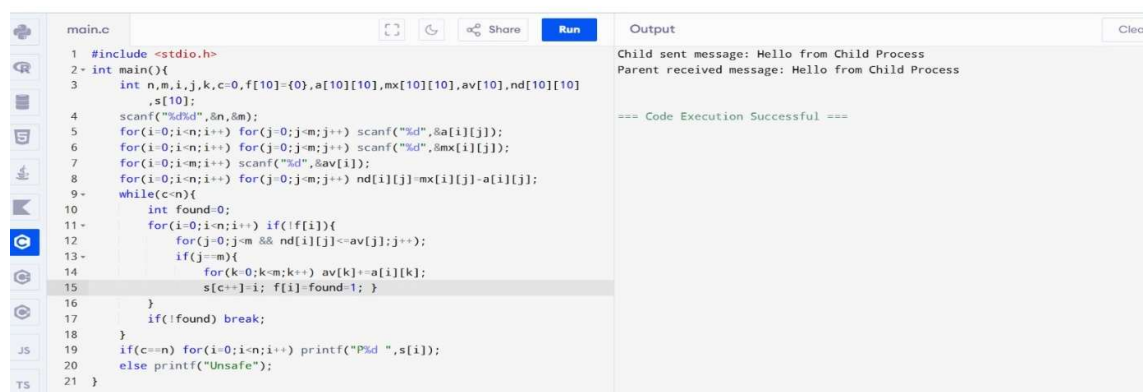
    if(!found)
        break;
}

if(c==n)
    for(i=0;i<n;i++)
        printf("P%d ",s[i]);
else
    printf("Unsafe");

return 0;
}

```

Output:



```

main.c
1 #include <stdio.h>
2 int main(){
3     int n,m,i,j,k,c=0,f[10]={0},a[10][10],mx[10][10],av[10],nd[10][10],s[10];
4     scanf("%d%d",&n,&m);
5     for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&a[i][j]);
6     for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&mx[i][j]);
7     for(i=0;i<n;i++) scanf("%d",&av[i]);
8     for(i=0;i<n;i++) for(j=0;j<m;j++) nd[i][j]=mx[i][j]-a[i][j];
9     while(c<n){
10        int found=0;
11        for(i=0;i<n;i++) if(!f[i]){
12            for(j=0;j<m && nd[i][j]<=av[j];j++);
13            if(j==m){
14                for(k=0;k<m;k++) av[k]+=a[i][k];
15                s[c++]=i; f[i]=found=1; }
16        }
17        if(!found) break;
18    }
19    if(c==n) for(i=0;i<n;i++) printf("P%d ",s[i]);
20    else printf("Unsafe");
21 }

```

Output

```

Child sent message: Hello from Child Process
Parent received message: Hello from Child Process

=== Code Execution Successful ===

```

Result:

Banker's Algorithm was successfully implemented, and the system is found to be in a safe state with the safe sequence:

P1 P3 P4 P0 P2

Experiment no:7

Aim:

To simulate the Producer-Consumer problem using semaphores in C.

Algorithm:

Step 1: Start

Step 2: Initialize empty semaphore $e = 1$ and full semaphore $f = 0$

Step 3: Create producer and consumer threads

Step 4: Producer waits for empty, produces item, then signals full

Step 5: Consumer waits for full, consumes item, then signals empty

Step 6: Wait for both threads to complete

Step 7: Stop

Source Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
```

```
sem_t e, f;
int x = 0;
```

```
void *p() {
    sem_wait(&e);
    x++;
    printf("P:%d\n", x);
    sem_post(&f);
}
```

```
void *c() {
    sem_wait(&f);
    printf("C:%d\n", x--);
    sem_post(&e);
}
```

```
int main() {
    pthread_t t1, t2;

    sem_init(&e, 0, 1);
    sem_init(&f, 0, 0);
```

```

pthread_create(&t1, 0, p, 0);
pthread_create(&t2, 0, c, 0);

pthread_join(t1, 0);
pthread_join(t2, 0);

return 0;
}

```

Output:

```

main.c
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 sem_t e,f; int x=0;
5 void *p(){ sem_wait(&e); x++; printf("P:%d\n",x); sem_post(&f); }
6 void *c(){ sem_wait(&f); printf("C:%d\n",x--); sem_post(&e); }
7 int main(){
8     pthread_t t1,t2; sem_init(&e,0,1); sem_init(&f,0,0);
9     pthread_create(&t1,0,p,0); pthread_create(&t2,0,c,0);
10    pthread_join(t1,0); pthread_join(t2,0);
11 }

```

Output

```

P:1
C:1

=== Code Execution Successful ===

```

Result:

The Producer-Consumer problem was successfully simulated using semaphores.

Experiment no: 8

Aim:

To implement process/thread synchronization using mutex locks in C.

Algorithm:

- Step 1: Start
- Step 2: Initialize shared variable $x = 0$
- Step 3: Initialize mutex lock
- Step 4: Create two threads
- Step 5: Each thread locks the mutex
- Step 6: Increment shared variable x
- Step 7: Display value of x
- Step 8: Unlock the mutex
- Step 9: Wait for both threads to finish
- Step 10: Destroy mutex and stop

Source Code:

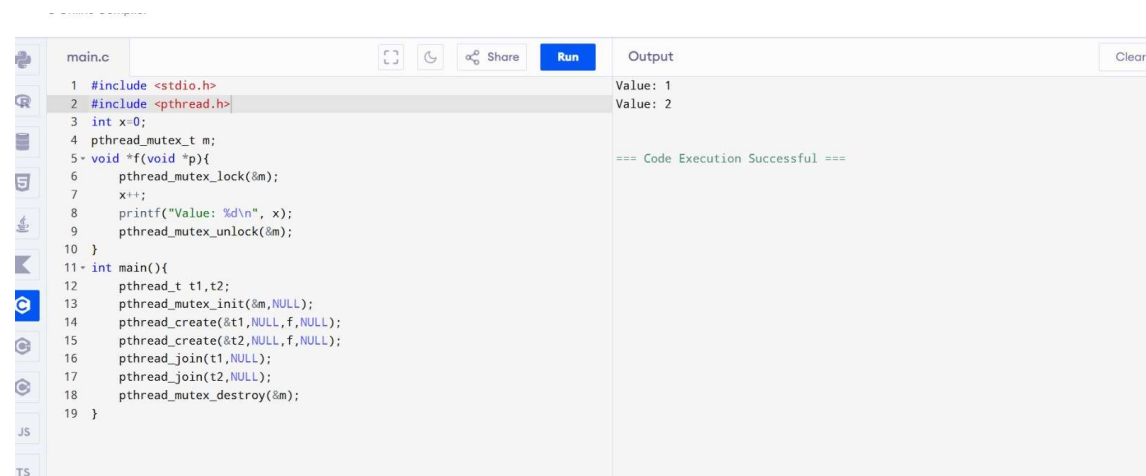
```
#include <stdio.h>
#include <pthread.h>

int x=0;
pthread_mutex_t m;

void *f(void *p){
    pthread_mutex_lock(&m);
    x++;
    printf("Value: %d\n", x);
    pthread_mutex_unlock(&m);
}

int main(){
    pthread_t t1,t2;
    pthread_mutex_init(&m,NULL);
    pthread_create(&t1,NULL,f,NULL);
    pthread_create(&t2,NULL,f,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    pthread_mutex_destroy(&m);
}
```

Output:



```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3 int x=0;
4 pthread_mutex_t m;
5 void *f(void *p){
6     pthread_mutex_lock(&m);
7     x++;
8     printf("Value: %d\n", x);
9     pthread_mutex_unlock(&m);
10 }
11 int main(){
12     pthread_t t1,t2;
13     pthread_mutex_init(&m,NULL);
14     pthread_create(&t1,NULL,f,NULL);
15     pthread_create(&t2,NULL,f,NULL);
16     pthread_join(t1,NULL);
17     pthread_join(t2,NULL);
18     pthread_mutex_destroy(&m);
19 }
```

Output

Value: 1
Value: 2

=== Code Execution Successful ===

Result:

Process synchronization using mutex locks was successfully implemented.

Experiment no:9

Aim:

To simulate the Reader-Writer problem using semaphores in C.

Algorithm:

- Step 1: Start
- Step 2: Initialize semaphores m and w
- Step 3: Create reader and writer threads
- Step 4: Reader increments read count
- Step 5: First reader locks writer semaphore
- Step 6: Reader performs reading
- Step 7: Reader decrements read count
- Step 8: Last reader releases writer semaphore
- Step 9: Writer locks semaphore, writes, then releases it
- Step 10: Stop

Source Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

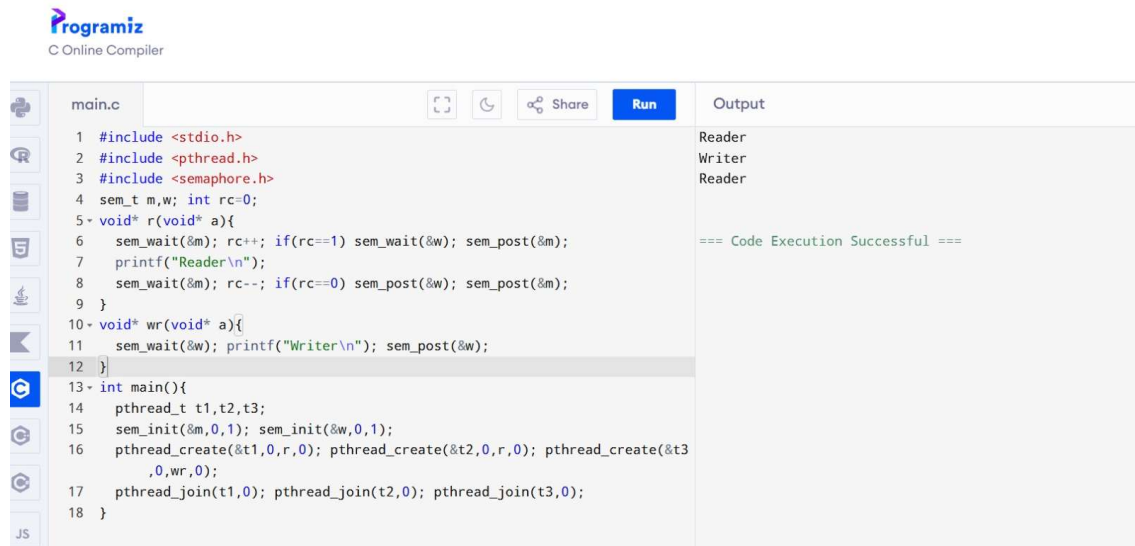
sem_t m,w; int rc=0;

void* r(void* a){
    sem_wait(&m); rc++; if(rc==1) sem_wait(&w); sem_post(&m);
    printf("Reader\n");
    sem_wait(&m); rc--; if(rc==0) sem_post(&w); sem_post(&m);
}

void* wr(void* a){
    sem_wait(&w); printf("Writer\n"); sem_post(&w);
}

int main(){
    pthread_t t1,t2,t3;
    sem_init(&m,0,1); sem_init(&w,0,1);
    pthread_create(&t1,0,r,0); pthread_create(&t2,0,r,0); pthread_create(&t3,0,wr,0);
    pthread_join(t1,0); pthread_join(t2,0); pthread_join(t3,0);
}
```

Output:



The screenshot displays the Programiz C Online Compiler interface. The code editor on the left contains a C program for simulating the Reader-Writer problem. The program includes headers for `<stdio.h>`, `<pthread.h>`, and `<semaphore.h>`. It defines a semaphore `m` and a counter `rc`. The `r` function (Reader) acquires the semaphore, increments `rc`, and prints "Reader". The `w` function (Writer) acquires the semaphore, prints "Writer", and releases it. The `main` function initializes the semaphore, creates three threads (two Readers and one Writer), and joins them. The output panel on the right shows the execution results: "Reader", "Writer", "Reader", and a success message "=== Code Execution Successful ===".

```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 sem_t m,w; int rc=0;
5 void* r(void* a){
6     sem_wait(&m); rc++; if(rc==1) sem_wait(&w); sem_post(&m);
7     printf("Reader\n");
8     sem_wait(&m); rc--; if(rc==0) sem_post(&w); sem_post(&m);
9 }
10 void* wr(void* a){
11     sem_wait(&w); printf("Writer\n"); sem_post(&w);
12 }
13 int main(){
14     pthread_t t1,t2,t3;
15     sem_init(&m,0,1); sem_init(&w,0,1);
16     pthread_create(&t1,0,r,0); pthread_create(&t2,0,r,0); pthread_create(&t3,0,wr,0);
17     pthread_join(t1,0); pthread_join(t2,0); pthread_join(t3,0);
18 }
```

Output

Reader
Writer
Reader

=== Code Execution Successful ===

Result:

Reader-Writer problem was successfully simulated using semaphores.

Experiment no :10

Aim:

To simulate the concept of Dining Philosophers problem using mutex locks in C.

Algorithm:

- Step 1: Start
- Step 2: Initialize 5 mutex locks for 5 forks
- Step 3: Create 5 philosopher threads
- Step 4: Each philosopher tries to pick left and right fork
- Step 5: Odd philosophers pick left first, even philosophers pick right first
- Step 6: Philosopher eats after getting both forks
- Step 7: Release both forks after eating
- Step 8: Wait for all philosopher threads to finish
- Step 9: Stop

Source Code:

```
#include <pthread.h>
#include <stdio.h>

#define N 5

pthread_mutex_t f[N];

void* phil(void* arg) {
    int i = *(int*)arg, l = i, r = (i + 1) % N;

    if(i % 2) {
        pthread_mutex_lock(&f[l]);
        pthread_mutex_lock(&f[r]);
    }
    else {
        pthread_mutex_lock(&f[r]);
        pthread_mutex_lock(&f[l]);
    }

    printf("Philosopher %d eating\n", i);

    pthread_mutex_unlock(&f[l]);
    pthread_mutex_unlock(&f[r]);
}
```

```

    return NULL;
}

int main() {
    pthread_t t[N];
    int a[N];

    for(int i = 0; i < N; i++)
        pthread_mutex_init(&f[i], NULL);

    for(int i = 0; i < N; i++) {
        a[i] = i;
        pthread_create(&t[i], NULL, phil, &a[i]);
    }

    for(int i = 0; i < N; i++)
        pthread_join(t[i], NULL);

    return 0;
}

```

Output:

```

main.c
1 #include <pthread.h>
2 #include <stdio.h>
3 #define N 5
4 pthread_mutex_t f[N];
5 void* phil(void* arg){
6     int i=*(int*)arg, l=i, r=(i+1)%N;
7     if(i%2){ pthread_mutex_lock(&f[l]); pthread_mutex_lock(&f[r]); }
8     else{ pthread_mutex_lock(&f[r]); pthread_mutex_lock(&f[l]); }
9     printf("Philosopher %d eating\n", i);
10    pthread_mutex_unlock(&f[l]);
11    pthread_mutex_unlock(&f[r]);
12    return NULL;
13 }
14 int main(){
15     pthread_t t[N]; int a[N];
16     for(int i=0;i<N;i++) pthread_mutex_init(&f[i],NULL);
17     for(int i=0;i<N;i++){ a[i]=i; pthread_create(&t[i],NULL,phil,&a[i]); }
18     for(int i=0;i<N;i++) pthread_join(t[i],NULL);
19 }

```

Output

```

Philosopher 0 eating
Philosopher 1 eating
Philosopher 2 eating
Philosopher 3 eating
Philosopher 4 eating

=== Code Execution Successful ===

```

Result:

Dining Philosophers problem was successfully simulated using mutex locks, and all philosophers were able to eat without deadlock.

Experiment No: 11

Aim:

To implement the First Fit Memory Management Algorithm using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of memory blocks
- Step 3: Enter the size of each memory block
- Step 4: Enter the number of processes
- Step 5: Enter the size of each process
- Step 6: Initialize allocation array as -1
- Step 7: Allocate each process to the first suitable memory block
- Step 8: Reduce the remaining memory size after allocation
- Step 9: Display process allocation details
- Step 10: Stop

Source Code:

```
#include <stdio.h>

int main() {
    int blockSize[10], processSize[10];
    int allocation[10];
    int nb, np;
    int i, j;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    printf("Enter size of each memory block:\n");
    for (i = 0; i < nb; i++) {
        scanf("%d", &blockSize[i]);
    }
}
```

```

printf("Enter number of processes: ");
scanf("%d", &np);

printf("Enter size of each process:\n");
for (i = 0; i < np; i++) {
    scanf("%d", &processSize[i]);
    allocation[i] = -1;
}

// First Fit Algorithm
for (i = 0; i < np; i++) {
    for (j = 0; j < nb; j++) {
        if (blockSize[j] >= processSize[i]) {
            allocation[i] = j;
            blockSize[j] = blockSize[j] - processSize[i];
            break;
        }
    }
}

printf("\nProcess No.\tProcess Size\tBlock No.\n");

for (i = 0; i < np; i++) {
    printf("%d\t%d\t\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

```

```

    }

    return 0;
}

```

Output:

The screenshot shows a C++ IDE with a file named `main.c`. The code implements the First Fit Memory Management Algorithm. It prompts the user to enter the number of memory blocks (5) and their sizes (100, 500, 200, 300, 600). It then prompts for the number of processes (4) and their sizes (212, 417, 112, 426). The algorithm attempts to allocate each process to the first available memory block of sufficient size. Process 1 (212) is allocated to Block 2 (500). Process 2 (417) is allocated to Block 5 (600). Process 3 (112) is allocated to Block 4 (300). Process 4 (426) is not allocated because no block is large enough. The output shows the allocation results in a table.

```

1 #include <stdio.h>
2
3 int main() {
4     int blockSize[10], processSize[10];
5     int allocation[10];
6     int nb, np;
7     int i, j;
8
9     printf("Enter number of memory blocks: ");
10    scanf("%d", &nb);
11
12    printf("Enter size of each memory block:\n");
13    for (i = 0; i < nb; i++) {
14        scanf("%d", &blockSize[i]);
15    }
16
17    printf("Enter number of processes: ");
18    scanf("%d", &np);
19
20    printf("Enter size of each process:\n");
21    for (i = 0; i < np; i++) {
22        scanf("%d", &processSize[i]);
23        allocation[i] = -1; // initially process is not allocated
24    }
25
26    // First Fit Algorithm
27    for (i = 0; i < np; i++) {
28        for (j = 0; j < nb; j++) {
29            if (blockSize[j] >= processSize[i]) {

```

Output:

```

Enter number of memory blocks: 5
Enter size of each memory block:
100
500
200
300
600
Enter number of processes: 4
Enter size of each process:
212
417
112
426

Process No. Process Size   Block No.
1          212           2
2          417           5
3          112           4
4          426         Not Allocated

=== Code Execution Successful ===

```

Result:

The First Fit Memory Management Algorithm was successfully implemented, and the processes were allocated to the first suitable memory blocks correctly.

Experiment no :12

Aim:

To implement the Best Fit Memory Management Algorithm using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of memory blocks
- Step 3: Enter the size of each memory block
- Step 4: Enter the number of processes
- Step 5: Enter the size of each process
- Step 6: Initialize allocation array as -1
- Step 7: Find the smallest suitable memory block for each process
- Step 8: Allocate process to the best suitable block
- Step 9: Reduce the remaining block size after allocation
- Step 10: Display process allocation details
- Step 11: Stop

Source Code:

```
#include <stdio.h>

int main() {
    int blockSize[10], processSize[10];
    int allocation[10];
    int nb, np;
    int i, j, bestIdx;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    printf("Enter size of each block:\n");
    for (i = 0; i < nb; i++) {
        scanf("%d", &blockSize[i]);
```

```

}

printf("Enter number of processes: ");
scanf("%d", &np);

printf("Enter size of each process:\n");
for (i = 0; i < np; i++) {
    scanf("%d", &processSize[i]);
    allocation[i] = -1;
}

// Best Fit Algorithm
for (i = 0; i < np; i++) {
    bestIdx = -1;

    for (j = 0; j < nb; j++) {
        if (blockSize[j] >= processSize[i]) {
            if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                bestIdx = j;
            }
        }
    }

    if (bestIdx != -1) {
        allocation[i] = bestIdx;
        blockSize[bestIdx] -= processSize[i];
    }
}

printf("\nProcess No\tProcess Size\tBlock No\n");

```

```

for (i = 0; i < np; i++) {
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

return 0;
}

```

Output:

```

main.c
1 #include <stdio.h>
2- int main() {
3     int blockSize[10], processSize[10];
4     int allocation[10];
5     int nb, np;
6     int i, j, bestIdx;
7     printf("Enter number of memory blocks: ");
8     scanf("%d", &nb);
9     printf("Enter size of each block:\n");
10-    for (i = 0; i < nb; i++) {
11        scanf("%d", &blockSize[i]);
12    }
13    printf("Enter number of processes: ");
14    scanf("%d", &np);
15
16    printf("Enter size of each process:\n");
17-    for (i = 0; i < np; i++) {
18        scanf("%d", &processSize[i]);
19        allocation[i] = -1;
20    }
21
22    // Best Fit Algorithm
23-    for (i = 0; i < np; i++) {
24        bestIdx = -1;
25-        for (j = 0; j < nb; j++) {
26-            if (blockSize[j] >= processSize[i]) {
27-                if (bestIdx == -1 || blockSize[j] <
                    blockSize[bestIdx]) {

```

Output

```

Enter number of memory blocks: 5
Enter size of each block:
100
500
200
300
600
Enter number of processes: 4
Enter size of each process:
212
417
112
426

Process No    Process Size    Block No
1             212            4
2             417            2
3             112            3
4             426            5

=== Code Execution Successful ===

```

Result:

The Best Fit Memory Management Algorithm was successfully implemented, and the processes were allocated to the most suitable memory blocks correctly.

Experiment No: 13

Aim:

To implement the Worst Fit Memory Management Algorithm using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of memory blocks
- Step 3: Enter the size of each memory block
- Step 4: Enter the number of processes
- Step 5: Enter the size of each process
- Step 6: Initialize allocation array as -1
- Step 7: Find the largest suitable memory block for each process
- Step 8: Allocate process to the worst suitable block
- Step 9: Reduce the remaining block size after allocation
- Step 10: Display process allocation details
- Step 11: Stop

Source Code:

```
#include <stdio.h>

int main() {
    int blockSize[10], processSize[10];
    int allocation[10];
    int nb, np;
    int i, j, worstIdx;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    printf("Enter size of each block:\n");
    for (i = 0; i < nb; i++) {
        scanf("%d", &blockSize[i]);
    }
```

```

printf("Enter number of processes: ");
scanf("%d", &np);

printf("Enter size of each process:\n");
for (i = 0; i < np; i++) {
    scanf("%d", &processSize[i]);
    allocation[i] = -1;
}

// Worst Fit Algorithm
for (i = 0; i < np; i++) {
    worstIdx = -1;

    for (j = 0; j < nb; j++) {
        if (blockSize[j] >= processSize[i]) {
            if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
                worstIdx = j;
            }
        }
    }

    if (worstIdx != -1) {
        allocation[i] = worstIdx;
        blockSize[worstIdx] -= processSize[i];
    }
}

printf("\nProcess No\tProcess Size\tBlock No\n");

```



```

    for (i = 0; i < np; i++) {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }

    return 0;
}

```

Output:

The screenshot shows a C program named 'main.c' being executed. The code implements the Worst Fit memory allocation algorithm. It prompts the user to enter the number of memory blocks (5), the size of each block (100, 500, 200, 300, 600), the number of processes (4), and the size of each process (212, 417, 112, 426). The output shows the allocation results for each process: Process 1 (size 212) is allocated to block 5; Process 2 (size 417) is allocated to block 2; Process 3 (size 112) is allocated to block 5; and Process 4 (size 426) is not allocated because no single block is large enough to accommodate it.

```

main.c
1 #include <stdio.h>
2
3- int main() {
4     int blockSize[10], processSize[10];
5     int allocation[10];
6     int nb, np;
7     int i, j, worstIdx;
8
9     printf("Enter number of memory blocks: ");
10    scanf("%d", &nb);
11
12    printf("Enter size of each block:\n");
13-    for (i = 0; i < nb; i++) {
14        scanf("%d", &blockSize[i]);
15    }
16
17    printf("Enter number of processes: ");
18    scanf("%d", &np);
19
20    printf("Enter size of each process:\n");
21-    for (i = 0; i < np; i++) {
22        scanf("%d", &processSize[i]);
23        allocation[i] = -1;
24    }
25
26    // Worst Fit Algorithm
27-    for (i = 0; i < np; i++) {
28        worstIdx = -1;

```

Output

```

Enter number of memory blocks: 5
Enter size of each block:
100
500
200
300
600
Enter number of processes: 4
Enter size of each process:
212
417
112
426

Process No  Process Size  Block No
1           212          5
2           417          2
3           112          5
4           426          Not Allocated

=== Code Execution Successful ===

```

Result:

The Worst Fit Memory Management Algorithm was successfully implemented, and the processes were allocated to the largest suitable memory blocks correctly.

Experiment No: 14

Aim:

To identify the system calls used to copy the contents of one file to another and implement them using a C program.

Algorithm:

- Step 1: Start
- Step 2: Open the source file in read mode using open()
- Step 3: Create/open the destination file using open()
- Step 4: Read data from source file using read()
- Step 5: Write data into destination file using write()
- Step 6: Repeat until end of file is reached
- Step 7: Close both files using close()
- Step 8: Display success message
- Step 9: Stop

Source Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int src, dest;
    char buffer[100];
    int bytes;

    // Open source file
    src = open("Source.txt", O_RDONLY);

    if (src < 0) {
        perror("Error opening source file");
```

```

        return 1;
    }

    // Create destination file

    dest = open("Dest.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest < 0) {

        perror("Error creating destination file");

        return 1;
    }

    // Copy content

    while ((bytes = read(src, buffer, sizeof(buffer))) > 0) {

        write(dest, buffer, bytes);
    }

    close(src);

    close(dest);

    printf("File copied successfully!\n");

    return 0;
}

```

Output:

The screenshot shows a C code editor with the following components:

- Editor:** Displays the C code for copying a file. The code includes headers for `stdio.h`, `fcntl.h`, and `unistd.h`. It defines a `main` function that opens `Source.txt` for reading and `Dest.txt` for writing. It then reads data from `Source.txt` into a buffer and writes it to `Dest.txt` until the end of the file is reached. Finally, it prints a success message and returns 0.
- Files Panel:** Located on the right, it shows the project files: `main.c`, `Source.txt`, and `Dest.txt`.
- Output Panel:** At the bottom, it shows the execution results:


```

$ gcc main.c -o main
$ ./main
File copied successfully!

Program finished with exit code 0
Press ENTER to exit console.

```

Result:

The contents of one file were successfully copied to another file using the system calls `open()`, `read()`, `write()`, and `close()`.

Experiment No: 15

Aim:

To write a C program to demonstrate thread related concepts such as create, join, equal, and exit using pthreads.

Algorithm:

- Step 1: Start
- Step 2: Include the required header files
- Step 3: Define a thread function
- Step 4: Get the current thread ID using pthread_self()
- Step 5: Create two threads using pthread_create()
- Step 6: Check whether two threads are equal using pthread_equal()
- Step 7: Wait for both threads to complete using pthread_join()
- Step 8: Exit from the thread using pthread_exit()
- Step 9: Display the output
- Step 10: Stop

Source Code:

```
#include <stdio.h>

#include <pthread.h>

void* threadFunction(void* arg) {
    pthread_t currentThread;
    currentThread = pthread_self();
    printf("Thread is running...\n");
    printf("Thread ID: %lu\n", currentThread);
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2;
    // Create threads
    pthread_create(&t1, NULL, threadFunction, NULL);
    pthread_create(&t2, NULL, threadFunction, NULL);
```

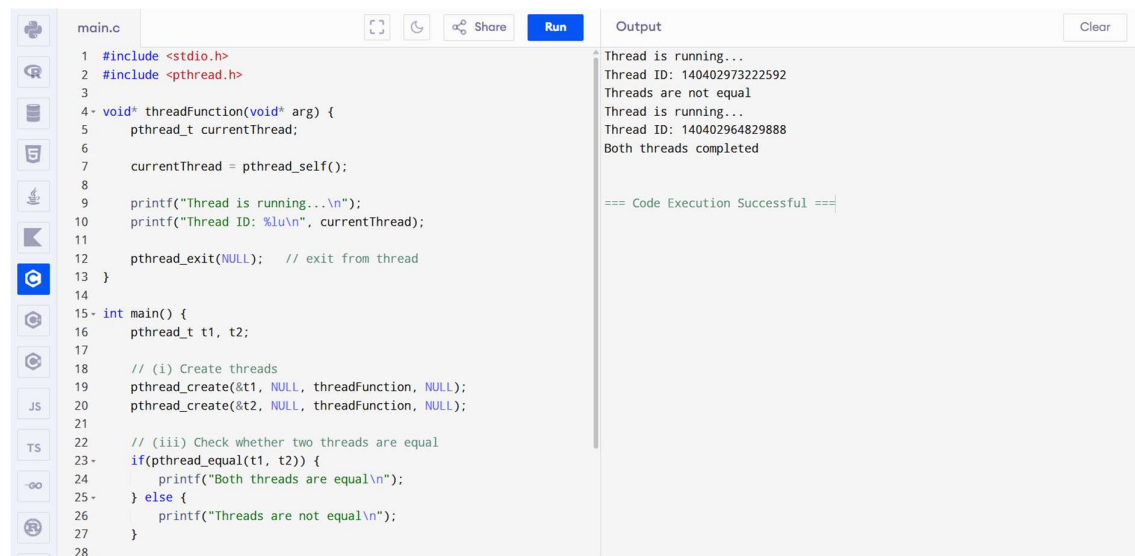
```

// Check whether two threads are equal
if(pthread_equal(t1, t2)) {
    printf("Both threads are equal\n");
} else {
    printf("Threads are not equal\n");
}

// Wait for threads to finish
pthread_join(t1, NULL);
pthread_join(t2, NULL);
printf("Both threads completed\n");
pthread_exit(NULL);
}

```

Output:



The screenshot shows a code editor with a file named 'main.c'. The code defines a thread function 'threadFunction' and a 'main' function. The 'main' function creates two threads, 't1' and 't2', using 'pthread_create'. It then checks if the two threads are equal using 'pthread_equal'. The output window on the right shows the execution results.

```

main.c
1 #include <stdio.h>
2 #include <pthread.h>
3
4 void* threadFunction(void* arg) {
5     pthread_t currentThread;
6
7     currentThread = pthread_self();
8
9     printf("Thread is running...\n");
10    printf("Thread ID: %lu\n", currentThread);
11
12    pthread_exit(NULL); // exit from thread
13 }
14
15 int main() {
16     pthread_t t1, t2;
17
18     // (i) Create threads
19     pthread_create(&t1, NULL, threadFunction, NULL);
20     pthread_create(&t2, NULL, threadFunction, NULL);
21
22     // (iii) Check whether two threads are equal
23     if(pthread_equal(t1, t2)) {
24         printf("Both threads are equal\n");
25     } else {
26         printf("Threads are not equal\n");
27     }
28 }

```

Output

```

Thread is running...
Thread ID: 140402973222592
Threads are not equal
Thread is running...
Thread ID: 140402964829888
Both threads completed

=== Code Execution Successful ===

```

Result:

The thread related concepts such as create, join, equal, and exit were successfully demonstrated using pthread functions.

Experiment No: 16

Aim:

To write a C program to organize files using a single level directory structure.

Algorithm:

Step 1: Start
Step 2: Display menu options: Create, Copy, and Delete
Step 3: Read the user choice
Step 4: If choice is 1, create a file using fopen()
Step 5: If choice is 2, open source and destination files using open()
Step 6: Copy contents using read() and write()
Step 7: If choice is 3, delete the file using remove()
Step 8: Display the appropriate message
Step 9: Stop

Source Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <fcntl.h>

#include <unistd.h>

#define BUFFER_SIZE 4096

void copy() {
    const char *sourcefile = "C:/Users/vivek balaji/OneDrive/Desktop/Source.txt";
    const char *destination_file = "C:/Users/vivek balaji/OneDrive/Desktop/Dest.txt";

    int source_fd = open(sourcefile, O_RDONLY);
    int dest_fd = open(destination_file, O_WRONLY | O_CREAT | O_TRUNC, 0666);

    char buffer[BUFFER_SIZE];
    ssize_t bytesRead, bytesWritten;
```

```

while ((bytesRead = read(source_fd, buffer, BUFFER_SIZE)) > 0) {
    bytesWritten = write(dest_fd, buffer, bytesRead);
}

close(source_fd);
close(dest_fd);

printf("File copied successfully.\n");
}

void create() {
    FILE *fp;

    fp = fopen("C:/Users/vivek balaji/OneDrive/Desktop/Source.txt", "w");

    if (fp != NULL) {
        printf("File created successfully\n");
        fclose(fp);
    }
}

int main() {
    int n;

    printf("1. Create \t2. Copy \t3. Delete\nEnter your choice: ");
    scanf("%d", &n);

    switch(n) {
        case 1:
            create();

```



```

        break;

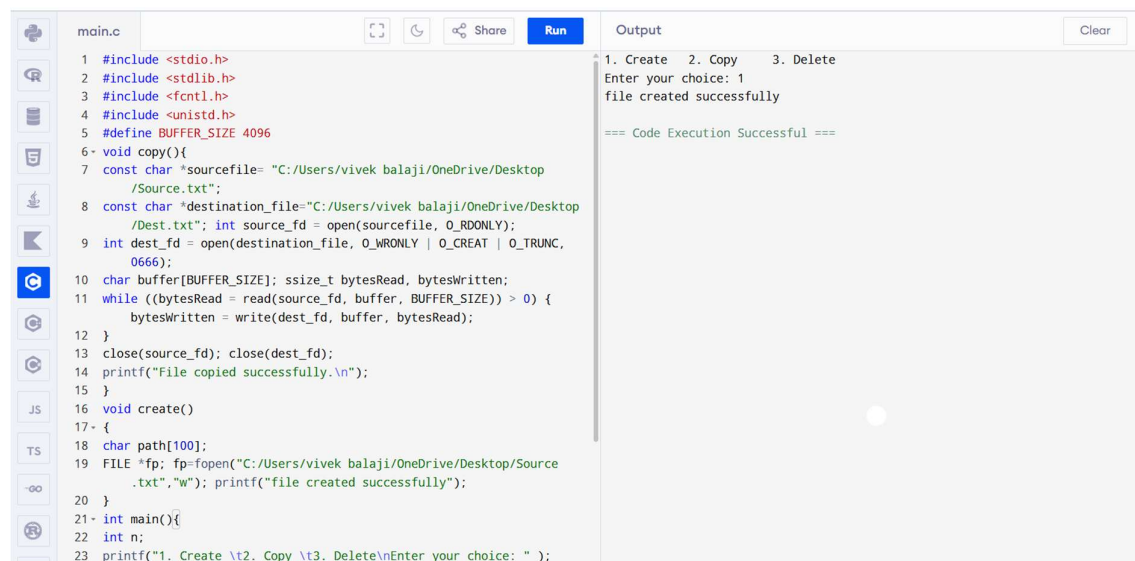
    case 2:
        copy();
        break;

    case 3:
        remove("C:/Users/vivek balaji/OneDrive/Desktop/Source.txt");
        printf("Deleted successfully\n");
        break;
}

return 0;
}

```

Output:



The screenshot shows a C++ IDE with a file named 'main.c'. The code implements a menu-driven file management system. The output window shows the program's execution: it displays a menu with '1. Create', '2. Copy', and '3. Delete', prompts the user to 'Enter your choice: 1', and then outputs 'file created successfully' followed by '=== Code Execution Successful ==='.

```

main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 #define BUFFER_SIZE 4096
6 void copy(){
7     const char *sourcefile= "C:/Users/vivek balaji/OneDrive/Desktop
   /Source.txt";
8     const char *destination_file="C:/Users/vivek balaji/OneDrive/Desktop
   /Dest.txt"; int source_fd = open(sourcefile, O_RDONLY);
9     int dest_fd = open(destination_file, O_WRONLY | O_CREAT | O_TRUNC,
   0666);
10    char buffer[BUFFER_SIZE]; ssize_t bytesRead, bytesWritten;
11    while ((bytesRead = read(source_fd, buffer, BUFFER_SIZE)) > 0) {
        bytesWritten = write(dest_fd, buffer, bytesRead);
12    }
13    close(source_fd); close(dest_fd);
14    printf("File copied successfully.\n");
15 }
16 void create()
17 {
18     char path[100];
19     FILE *fp; fp=fopen("C:/Users/vivek balaji/OneDrive/Desktop/Source
   .txt","w"); printf("file created successfully");
20 }
21 int main(){
22     int n;
23     printf("1. Create \t2. Copy \t3. Delete\nEnter your choice: " );

```

Output

```

1. Create 2. Copy 3. Delete
Enter your choice: 1
file created successfully

=== Code Execution Successful ===

```

Result:

The single level directory file organization was successfully implemented, and file operations such as create, copy, and delete were performed correctly.

Experiment No: 17

Aim:

To simulate the First Come First Served (FCFS) Disk Scheduling Algorithm using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of disk requests
- Step 3: Enter the disk request sequence
- Step 4: Enter the initial head position
- Step 5: Move the disk head according to FCFS order
- Step 6: Calculate total head movement
- Step 7: Display the disk scheduling order
- Step 8: Display total head movement
- Step 9: Stop

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n, head, i;
    int request[20];
    int total_movement = 0;
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter disk request sequence:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &request[i]);
    }
}
```

```

printf("Enter initial head position: ");
scanf("%d", &head);

printf("\nFCFS Disk Scheduling Order:\n");
printf("%d", head);
for(i = 0; i < n; i++)
{
    total_movement += abs(request[i] - head);
    head = request[i];
    printf(" -> %d", head);
}

printf("\n\nTotal Head Movement = %d\n", total_movement);
return 0;
}

```

Output:

```

main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int n, head, i;
7     int request[20];
8     int total_movement = 0;
9
10    printf("Enter number of disk requests: ");
11    scanf("%d", &n);
12
13    printf("Enter disk request sequence:\n");
14    for(i = 0; i < n; i++)
15    {
16        scanf("%d", &request[i]);
17    }
18
19    printf("Enter initial head position: ");
20    scanf("%d", &head);
21
22    printf("\nFCFS Disk Scheduling Order:\n");
23    printf("%d", head);
24
25    for(i = 0; i < n; i++)
26    {
27        total_movement += abs(request[i] - head);
28        head = request[i];

```

Output

```

Enter number of disk requests: 5
Enter disk request sequence:
98
183
37
122
14
Enter initial head position: 53

FCFS Disk Scheduling Order:
53 -> 98 -> 183 -> 37 -> 122 -> 14

Total Head Movement = 469

=== Code Execution Successful ===

```

Result:

The First Come First Served (FCFS) Disk Scheduling Algorithm was successfully simulated, and the total head movement was calculated correctly.

Experiment No: 18

Aim:

To simulate the SCAN Disk Scheduling Algorithm using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of disk requests
- Step 3: Enter the disk request sequence
- Step 4: Enter the initial head position
- Step 5: Enter disk size and direction
- Step 6: Sort the disk request sequence
- Step 7: Move the head in the given direction and service requests
- Step 8: After reaching one end, reverse the direction
- Step 9: Calculate total head movement
- Step 10: Display SCAN order and total head movement
- Step 11: Stop

Source Code:

```
#include <stdio.h>

#include <stdlib.h>

int main()
{
    int n, head, size, direction;
    int request[20], i, j, temp;
    int total_movement = 0;
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request sequence:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &request[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size: ");
```

```

scanf("%d", &size);
printf("Enter direction (0 = left, 1 = right): ");
scanf("%d", &direction);
// Sort requests
for(i = 0; i < n; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(request[i] > request[j])
        {
            temp = request[i];
            request[i] = request[j];
            request[j] = temp;
        }
    }
}
printf("\nSCAN Order:\n");
if(direction == 1)
{
    for(i = 0; i < n; i++)
    {
        if(request[i] >= head)
        {
            printf("%d -> ", request[i]);
            total_movement += abs(request[i] - head);
            head = request[i];
        }
    }
    total_movement += abs((size - 1) - head);
    head = size - 1;
}

```

```

for(i = n - 1; i >= 0; i--)
{
    if(request[i] < head)
    {
        printf("%d -> ", request[i]);
        total_movement += abs(request[i] - head);
        head = request[i];
    }
}
else
{
    for(i = n - 1; i >= 0; i--)
    {
        if(request[i] <= head)
        {
            printf("%d -> ", request[i]);
            total_movement += abs(request[i] - head);
            head = request[i];
        }
    }

    total_movement += abs(head - 0);
    head = 0;
    for(i = 0; i < n; i++)
    {
        if(request[i] > head)
        {
            printf("%d -> ", request[i]);

```

```

        total_movement += abs(request[i] - head);

        head = request[i];
    }

}

printf("\nTotal Head Movement = %d\n", total_movement);

return 0;
}

```

Output:

```

main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int n, head, size, direction;
7      int request[20], i, j, temp;
8      int total_movement = 0;
9
10     printf("Enter number of requests: ");
11     scanf("%d", &n);
12
13     printf("Enter request sequence:\n");
14     for(i = 0; i < n; i++)
15         scanf("%d", &request[i]);
16
17     printf("Enter initial head position: ");
18     scanf("%d", &head);
19
20     printf("Enter disk size: ");
21     scanf("%d", &size);
22
23     printf("Enter direction (0 = left, 1 = right): ");
24     scanf("%d", &direction);
25
26     // Sort requests
27     for(i = 0; i < n; i++)
28     {

```

```

Output
Enter number of requests: 5
Enter request sequence:
98
183
37
122
4
Enter initial head position: 53
Enter disk size: 200
Enter direction (0 = left, 1 = right): 1

SCAN Order:
98 -> 122 -> 183 -> 183 -> 122 -> 98 -> 37 -> 4 ->
Total Head Movement = 341

=== Code Execution Successful ===

```

Result:

The SCAN Disk Scheduling Algorithm was successfully simulated, and the total head movement was calculated correctly.

Experiment No: 19

Aim:

To simulate the C-SCAN Disk Scheduling Algorithm using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of disk requests
- Step 3: Enter the disk request sequence
- Step 4: Enter the initial head position
- Step 5: Enter the disk size
- Step 6: Sort the disk request sequence
- Step 7: Service all requests greater than or equal to the head position
- Step 8: Move the head to the end of the disk
- Step 9: Jump the head to the beginning of the disk
- Step 10: Service the remaining requests
- Step 11: Calculate and display total head movement
- Step 12: Stop

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, size, i, j, temp;
    int request[20];
    int total_movement = 0;
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request sequence:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &request[i]);

    printf("Enter initial head position: ");
```



```

scanf("%d", &head);
printf("Enter disk size: ");
scanf("%d", &size);
// Sort requests
for(i = 0; i < n; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(request[i] > request[j])
        {
            temp = request[i];
            request[i] = request[j];
            request[j] = temp;
        }
    }
}
printf("\nC-SCAN Order:\n");
// Move right
for(i = 0; i < n; i++)
{
    if(request[i] >= head)
    {
        printf("%d -> ", request[i]);
        total_movement += abs(request[i] - head);
        head = request[i];
    }
}

// Go to end of disk
total_movement += abs((size - 1) - head);

```

```

head = size - 1;

// Jump to beginning

total_movement += abs(head - 0);

head = 0;

// Continue servicing remaining requests

for(i = 0; i < n; i++)
{
    if(request[i] < head)
    {
        printf("%d -> ", request[i]);

        total_movement += abs(request[i] - head);

        head = request[i];
    }
}

printf("\nTotal Head Movement = %d\n", total_movement)

return 0;
}

```

Output:

The screenshot shows a C program in a code editor with a 'Run' button. The program implements the C-SCAN disk scheduling algorithm. The output window displays the following text:

```

Enter number of requests: 5
Enter request sequence:
98
183
37
122
14
Enter initial head position: 53
Enter disk size: 200

C-SCAN Order:
98 -> 122 -> 183 ->
Total Head Movement = 345

=== Code Execution Successful ===

```

Result:

The C-SCAN Disk Scheduling Algorithm was successfully simulated, and the total head movement was calculated correctly.

Experiment No: 20

Aim:

To simulate the First In First Out (FIFO) Paging Technique of memory management using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of pages
- Step 3: Enter the page reference string
- Step 4: Enter the number of frames
- Step 5: Initialize all frames as -1
- Step 6: Check whether the current page is already present in frame
- Step 7: If page is not present, replace the oldest page using FIFO method
- Step 8: Increase the page fault count
- Step 9: Display the frame status for each page
- Step 10: Display total page faults
- Step 11: Stop

Source Code:

```
#include <stdio.h>

int main()
{
    int pages[50], frame[10];
    int n, f, i, j, k, page_faults = 0;
    int index = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);
```

```

// Initialize frames with -1
for(i = 0; i < f; i++)
    frame[i] = -1;
printf("\nPage\tFrames\n");
for(i = 0; i < n; i++)
{
    found = 0;

    // Check if page already in frame
    for(j = 0; j < f; j++)
    {
        if(frame[j] == pages[i])
        {
            found = 1;
            break;
        }
    }
    // If not found, page fault occurs
    if(found == 0)
    {
        frame[index] = pages[i];
        index = (index + 1) % f;
        page_faults++;
    }
    // Print frames
    printf("%d\t", pages[i]);
    for(k = 0; k < f; k++)
        printf("%d ", frame[k]);
    printf("\n");
}

```

```

printf("\nTotal Page Faults = %d\n", page_faults);

return 0;

}

```

Output:

```

main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int pages[50], frame[10];
6     int n, f, i, j, k, page_faults = 0;
7     int index = 0, found;
8
9     printf("Enter number of pages: ");
10    scanf("%d", &n);
11
12    printf("Enter page reference string:\n");
13    for(i = 0; i < n; i++)
14        scanf("%d", &pages[i]);
15
16    printf("Enter number of frames: ");
17    scanf("%d", &f);
18
19    // Initialize frames with -1
20    for(i = 0; i < f; i++)
21        frame[i] = -1;
22
23    printf("\nPage\tFrames\n");
24
25    for(i = 0; i < n; i++)
26    {
27        found = 0;

```

Output

```

Enter number of pages: 8
Enter page reference string:
1
2
3
4
1
2
5
1
Enter number of frames: 3

Page   Frames
1   1  -1  -1
2   1  2  -1
3   1  2  3
4   4  2  3
1   4  1  3
2   4  1  2
5   5  1  2
1   5  1  2

Total Page Faults = 7

=== Code Execution Successful ===

```

Result:

The First In First Out (FIFO) Paging Technique was successfully simulated, and the total number of page faults was calculated correctly.

Experiment No: 21

Aim:

To simulate the Least Recently Used (LRU) Paging Technique of memory management using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of pages
- Step 3: Enter the page reference string
- Step 4: Enter the number of frames
- Step 5: Initialize frames and time arrays
- Step 6: Check whether the page is already available in frame
- Step 7: If page exists, update its recent usage time
- Step 8: If page fault occurs, replace the least recently used page
- Step 9: Display frame contents after each page reference
- Step 10: Count total page faults
- Step 11: Display total page faults
- Step 12: Stop

Source Code:

```
#include <stdio.h>

int main()
{
    int pages[50], frame[10], time[10];
    int n, f, i, j, k, page_faults = 0;
    int count = 0, found, pos, min;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
```

```

// Initialize frames
for(i = 0; i < f; i++)
{
    frame[i] = -1;
    time[i] = 0;
}
printf("\nPage\tFrames\n");
for(i = 0; i < n; i++)
{
    found = 0;
    // Check if page already exists
    for(j = 0; j < f; j++)
    {
        if(frame[j] == pages[i])
        {
            found = 1;
            count++;
            time[j] = count;
            break;
        }
    }
    // Page fault
    if(found == 0)
    {
        min = time[0];
        pos = 0;
        // Find least recently used
        for(j = 1; j < f; j++)
        {
            if(time[j] < min)

```

```

        {
            min = time[j];
            pos = j;
        }
    }

    frame[pos] = pages[i];
    count++;
    time[pos] = count;
    page_faults++;
}

// Print frames
printf("%d\t", pages[i]);
for(k = 0; k < f; k++)
    printf("%d ", frame[k]);
printf("\n");
}

printf("\nTotal Page Faults = %d\n", page_faults);
return 0;
}

```

Output:

The screenshot shows a C program being executed in a terminal. The program simulates the Least Recently Used (LRU) paging technique. It prompts the user for the number of pages (8) and a page reference string (1 2 3 4 1 2 5 1). It then prompts for the number of frames (3). The program displays a table of the page frames as they change over time, showing page faults when a new page is loaded into a full frame. Finally, it prints the total number of page faults, which is 7.

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int pages[50], frame[10], time[10];
6      int n, f, i, j, k, page_faults = 0;
7      int count = 0, found, pos, min;
8
9      printf("Enter number of pages: ");
10     scanf("%d", &n);
11
12     printf("Enter page reference string:\n");
13     for(i = 0; i < n; i++)
14         scanf("%d", &pages[i]);
15
16     printf("Enter number of frames: ");
17     scanf("%d", &f);
18
19     // Initialize frames
20     for(i = 0; i < f; i++)
21     {
22         frame[i] = -1;
23         time[i] = 0;
24     }
25
26     printf("\nPage\tFrames\n");
27     for(i = 0; i < n; i++)

```

```

Enter number of pages: 8
Enter page reference string:
1
2
3
4
1
2
5
1
Enter number of frames: 3

Page    Frames
1      1 -1 -1
2      1 2 -1
3      1 2 3
4      4 2 3
1      4 1 3
2      4 1 2
5      5 1 2
1      5 1 2

Total Page Faults = 7

=== Code Execution Successful ===

```

Result:

The Least Recently Used (LRU) Paging Technique was successfully simulated, and the total number of page faults was calculated correctly.

Experiment No: 22

Aim:

To simulate the Sequential File Allocation Strategy using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of records
- Step 3: Enter the records of the file
- Step 4: Enter the record position to access
- Step 5: Check whether the record position is valid
- Step 6: Read all previous records sequentially
- Step 7: Display the required record
- Step 8: Stop

Source Code:

```
#include <stdio.h>

int main()
{
    int n, i, search;

    printf("Enter number of records: ");
    scanf("%d", &n);

    int record[n];

    printf("Enter records:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &record[i]);
    }

    printf("Enter record position to access: ");
    scanf("%d", &search);

    if(search < 1 || search > n)
    {
        printf("Invalid record position!\n");
        return 0;
    }

    printf("\nReading records sequentially:\n");
```

```

for(i = 0; i < search; i++)
{
    printf("Record %d = %d\n", i + 1, record[i]);
}

printf("\nRequired record found: Record %d = %d\n",
       search, record[search - 1]);

return 0;
}

```

Output:

```

main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int n, i, search;
6
7      printf("Enter number of records: ");
8      scanf("%d", &n);
9
10     int record[n];
11
12     printf("Enter records:\n");
13     for(i = 0; i < n; i++)
14     {
15         scanf("%d", &record[i]);
16     }
17
18     printf("Enter record position to access: ");
19     scanf("%d", &search);
20
21     if(search < 1 || search > n)
22     {
23         printf("Invalid record position!\n");
24         return 0;
25     }
26
27     printf("\nReading records sequentially:\n");

```

Output

```

Enter number of records: 5
Enter records:
10
20
30
40
50
Enter record position to access: 4

Reading records sequentially:
Record 1 = 10
Record 2 = 20
Record 3 = 30
Record 4 = 40

Required record found: Record 4 = 40

=== Code Execution Successful ===

```

Result:

The Sequential File Allocation Strategy was successfully simulated, and the required record was accessed by reading all previous records sequentially.

Experiment No: 23

Aim:

To simulate the Indexed File Allocation Strategy using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of file blocks
- Step 3: Enter the block numbers into the index block
- Step 4: Display the index block entries
- Step 5: Enter the block number to access
- Step 6: Check whether the block number is valid
- Step 7: Display the disk location of the required block
- Step 8: Stop

Source Code:

```
#include <stdio.h>

int main()
{
    int n, i, indexBlock[20], choice;

    printf("Enter number of file blocks: ");
    scanf("%d", &n);

    printf("Enter block numbers (index block):\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &indexBlock[i]);
    }

    printf("\nIndex Block Entries:\n");
    for(i = 0; i < n; i++)
    {
        printf("Block %d -> %d\n", i + 1, indexBlock[i]);
    }

    printf("\nEnter block number to access: ");
    scanf("%d", &choice);

    if(choice < 1 || choice > n)
    {
        printf("Invalid block!\n");
    }
}
```

```

    }
else
{
    printf("Block %d is stored at disk location %d\n",
           choice, indexBlock[choice - 1]);
}
return 0;
}

```

Output:

```

main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, indexBlock[20], choice;
6
7     printf("Enter number of file blocks: ");
8     scanf("%d", &n);
9
10    printf("Enter block numbers (index block):\n");
11    for(i = 0; i < n; i++)
12    {
13        scanf("%d", &indexBlock[i]);
14    }
15
16    printf("\nIndex Block Entries:\n");
17    for(i = 0; i < n; i++)
18    {
19        printf("Block %d -> %d\n", i + 1, indexBlock[i]);
20    }
21
22    printf("\nEnter block number to access: ");
23    scanf("%d", &choice);
24
25    if(choice < 1 || choice > n)
26    {
27        printf("Invalid block!\n");
28    }

```

Output

```

Enter number of file blocks: 5
Enter block numbers (index block):
11
24
36
48
59

Index Block Entries:
Block 1 -> 11
Block 2 -> 24
Block 3 -> 36
Block 4 -> 48
Block 5 -> 59

Enter block number to access: 3
Block 3 is stored at disk location 36

=== Code Execution Successful ===

```

Result:

The Indexed File Allocation Strategy was successfully simulated, and the required file block was accessed correctly using the index block.

Experiment No: 24

Aim:

To simulate the Linked File Allocation Strategy using a C program.

Algorithm:

- Step 1: Start
- Step 2: Enter the number of file blocks
- Step 3: Enter the disk block numbers
- Step 4: Link each block to the next block
- Step 5: Display the linked allocation structure
- Step 6: Display the first and last block information
- Step 7: Stop

Source Code:

```
#include <stdio.h>

int main()
{
    int n, i;
    int block[20];

    printf("Enter number of file blocks: ");
    scanf("%d", &n);

    printf("Enter disk block numbers:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &block[i]);
    }

    printf("\nLinked Allocation:\n")
    for(i = 0; i < n - 1; i++)
    {
        printf("Block %d -> points to Block %d\n", block[i], block[i + 1]);
    }

    printf("Block %d -> NULL\n", block[n - 1]);

    printf("\nDirectory Information:\n");
    printf("First Block = %d\n", block[0]);
    printf("Last Block = %d\n", block[n - 1]);
}
```

```

    return 0;
}

```

Output:

main.c	Output
<pre> 1 #include <stdio.h> 2 3 int main() 4 { 5 int n, i; 6 int block[20]; 7 8 printf("Enter number of file blocks: "); 9 scanf("%d", &n); 10 11 printf("Enter disk block numbers:\n"); 12 for(i = 0; i < n; i++) 13 { 14 scanf("%d", &block[i]); 15 } 16 17 printf("\nLinked Allocation:\n"); 18 19 for(i = 0; i < n - 1; i++) 20 { 21 printf("Block %d -> points to Block %d\n", block[i], block[i + 1]); 22 } 23 24 printf("Block %d -> NULL\n", block[n - 1]); 25 26 printf("\nDirectory Information:\n"); 27 printf("First Block = %d\n", block[0]); </pre>	<pre> Enter number of file blocks: 5 Enter disk block numbers: 10 25 40 18 60 Linked Allocation: Block 10 -> points to Block 25 Block 25 -> points to Block 40 Block 40 -> points to Block 18 Block 18 -> points to Block 60 Block 60 -> NULL Directory Information: First Block = 10 Last Block = 60 === Code Execution Successful === </pre>

Result:

The Linked File Allocation Strategy was successfully simulated, and the file blocks were connected using pointers from one block to the next.